Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 5.2: Python Sets

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Create a Set
2. Accessing Items
3. Editing Items (add & update)
4. Delete Items
5. Set Operations
6. Set Functions
7. Frozenset
8. Set Comprehension

48 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 5.2: Python Sets - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Create a Set, Accessing Items, Editing Items (add & update), Delete Items, Set Operations, Set Functions, Frozenset, Set Comprehension
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Create a Set

Q1. Unique Visitors

BEGINNER

A website logged visitor IDs 101, 102, 101, 103, 102, 104. Store them in a set to keep only unique visitors. Print the set and the number of unique visitors. (Note: the printed order of a set may differ on your computer.)

Solution:

```
visitors = {101, 102, 101, 103, 102, 104}

print(visitors)
print(len(visitors))
```

Output:

```
{104, 101, 102, 103}
4
```

Q2. Set from a List

BEGINNER

A list of courses selected by students is ["Python", "SQL", "Python", "Excel", "SQL"]. Convert it to a set and print the sorted unique courses.

Solution:

```
courses = ["Python", "SQL", "Python", "Excel", "SQL"]

unique_courses = set(courses)
print(sorted(unique_courses))
print(len(unique_courses))
```

Output:

```
['Excel', 'Python', 'SQL']
3
```

Q3. Empty Set Trap

INTERMEDIATE

Create an empty set for storing blocked users. Show that {} creates a dictionary, while set() creates an empty set.

Solution:

```
a = {}
b = set()

print(type(a))
print(type(b))
print(len(b))
```

Output:

```
<class 'dict'>
<class 'set'>
0
```

Q4. Unique Letters in a Word

INTERMEDIATE

Find the unique letters in the word "MISSISSIPPI" using set() and print them in sorted order with their count.

Solution:

```
word = "MISSISSIPPI"

letters = set(word)
print(sorted(letters))
print(len(letters))
```

Output:

```
['I', 'M', 'P', 'S']
4
```

Q5. Only Immutable Items Allowed

ADVANCED

Try to create a set containing numbers, a string and a tuple: {1, "Pune", (18.5, 73.8)}. Then try adding a list inside try-except and print the error.

Solution:

```
valid = {1, "Pune", (18.5, 73.8)}
print(len(valid))

try:
    invalid = {1, [2, 3]}
except TypeError as e:
    print(e)
```

Output:

```
3
unhashable type: 'list'
```

Q6. True and 1 are Same in a Set

ADVANCED

Create the set {1, True, 1.0, 0, False}. Predict and print its length. Explain in a comment why True and 1 count as the same item.

Solution:

```
values = {1, True, 1.0, 0, False}

print(len(values))
print(values)
# True == 1 and False == 0 with the same hash,
# so the set keeps only one of each
```

Output:

```
2
{0, 1}
```

2. Accessing Items

Q7. No Indexing in Sets

BEGINNER

Sets are unordered. Try to access the first item of {"Red", "Blue"} using [0] inside try-except and print the error.

Solution:

```
colours = {"Red", "Blue"}

try:
    print(colours[0])
except TypeError as e:
    print(e)
```

Output:

```
'set' object is not subscriptable
```

Q8. Loop Through a Set

BEGINNER

Print every unique pin code from {411001, 411014, 411038} in sorted order using a for loop.

Solution:

```
pin_codes = {411001, 411014, 411038}

for pin in sorted(pin_codes):
    print(pin)
```

Output:

```
411001
411014
411038
```

Q9. Check Membership

INTERMEDIATE

Registered emails are {"a@x.com", "b@x.com", "c@x.com"}. Check whether "b@x.com" is registered and whether "z@x.com" is not registered.

Solution:

```
registered = {"a@x.com", "b@x.com", "c@x.com"}

print("b@x.com" in registered)
print("z@x.com" not in registered)
```

Output:

```
True
True
```

Q10. Access via Conversion

INTERMEDIATE

To read items by position, convert the set {50, 10, 30, 20} into a sorted list. Print the smallest and the largest value by index.

Solution:

```
scores = {50, 10, 30, 20}

ordered = sorted(scores)
print(ordered)
print(ordered[0])
print(ordered[-1])
```

Output:

```
[10, 20, 30, 50]
10
50
```

Q11. Fast Lookup with Sets

ADVANCED

Compare lookup time of 99999 in a list vs a set containing numbers 0 to 99999 using timeit (1000 checks). Print whether the set is faster.

Solution:

```
import timeit

numbers_list = list(range(100000))
numbers_set = set(numbers_list)

t_list = timeit.timeit(lambda: 99999 in numbers_list, number=1000)
t_set = timeit.timeit(lambda: 99999 in numbers_set, number=1000)

print(t_set < t_list)
```

Sample Output (changes on every run):

```
True
```

Q12. Check for Allowed Characters

ADVANCED

A username may only contain lowercase letters and digits. Using a set of allowed characters, check the usernames "ravi2026" and "Ravi@2026".

Solution:

```
allowed = set("abcdefghijklmnopqrstuvwxyz0123456789")

for user in ["ravi2026", "Ravi@2026"]:
    ok = set(user) <= allowed
    print(user, ok)
```

Output:

```
ravi2026 True
Ravi@2026 False
```

3. Editing Items (add & update)

Q13. Add One Tag

BEGINNER

A blog post has tags {"python", "coding"}. Add the tag "beginner" using add() and print the sorted tags.

Solution:

```
tags = {"python", "coding"}

tags.add("beginner")
print(sorted(tags))
```

Output:

```
['beginner', 'coding', 'python']
```

Q14. Adding a Duplicate

BEGINNER

Add "python" again to the tags set {"python", "coding"}. Print the length before and after to show duplicates are ignored.

Solution:

```
tags = {"python", "coding"}
print(len(tags))

tags.add("python")
print(len(tags))
```

Output:

```
2
2
```

Q15. Add Many Items with update()

INTERMEDIATE

Skills of an employee are {"Excel"}. Add ["SQL", "Power BI", "Excel"] using update() and print the sorted skills.

Solution:

```
skills = {"Excel"}

skills.update(["SQL", "Power BI", "Excel"])
print(sorted(skills))
```

Output:

```
['Excel', 'Power BI', 'SQL']
```

Q16. No Direct Editing

INTERMEDIATE

Sets cannot change an item in place. Replace "Nasik" with the correct spelling "Nashik" in {"Pune", "Nasik"} by removing the old value and adding the new one.

Solution:

```
cities = {"Pune", "Nasik"}

cities.remove("Nasik")
cities.add("Nashik")
print(sorted(cities))
```

Output:

```
['Nashik', 'Pune']
```

Q17. Collect Unique Words from Reviews

ADVANCED

Reviews are ["good food", "good service", "bad parking"]. Use update() with split() in a loop to collect all unique words.

Solution:

```
reviews = ["good food", "good service", "bad parking"]
words = set()

for r in reviews:
    words.update(r.split())

print(sorted(words))
print(len(words))
```

Output:

```
['bad', 'food', 'good', 'parking', 'service']
5
```

Q18. Track New Logins

ADVANCED

Logins arrive in this order: ["ravi", "sana", "ravi", "om", "sana", "jay"]. Use a set to print a user only the FIRST time they log in.

Solution:

```
logins = ["ravi", "sana", "ravi", "om", "sana", "jay"]
seen = set()

for user in logins:
    if user not in seen:
        print("New:", user)
        seen.add(user)

print(len(seen))
```

Output:

```
New: ravi
New: sana
New: om
New: jay
4
```

4. Delete Items

Q19. remove() a Member

BEGINNER

Gym members are {"Amit", "Bina", "Chirag"}. Remove "Bina" using remove().

Solution:

```
members = {"Amit", "Bina", "Chirag"}

members.remove("Bina")
print(sorted(members))
```

Output:

```
['Amit', 'Chirag']
```

Q20. discard() Safely

BEGINNER

Use discard() to remove "Zara" (not present) and "Amit" (present) from {"Amit", "Chirag"}. Show that discard() gives no error for a missing item.

Solution:

```
members = {"Amit", "Chirag"}

members.discard("Zara")
members.discard("Amit")
print(members)
```

Output:

```
{'Chirag'}
```

Q21. remove() Error

INTERMEDIATE

Try to remove "Zara" from {"Amit"} using remove() inside try-except and print the KeyError message.

Solution:

```
members = {"Amit"}

try:
    members.remove("Zara")
except KeyError as e:
    print("KeyError:", e)
```

Output:

```
KeyError: 'Zara'
```

Q22. pop() and clear()

INTERMEDIATE

A set of pending tasks is {"T1", "T2", "T3"}. pop() one task (any one), print how many remain, then clear() the set.

Solution:

```
tasks = {"T1", "T2", "T3"}

done = tasks.pop()
print(len(tasks))

tasks.clear()
print(tasks)
```

Output:

```
2
set()
```

Q23. Remove Blocked Numbers

ADVANCED

Contacts are {9811, 9822, 9833, 9844} and blocked numbers are {9822, 9844, 9999}. Remove all blocked numbers from contacts using difference_update().

Solution:

```
contacts = {9811, 9822, 9833, 9844}
blocked = {9822, 9844, 9999}

contacts.difference_update(blocked)
print(sorted(contacts))
```

Output:

```
[9811, 9833]
```

Q24. Delete Items While Looping (Safely)

ADVANCED

From the set of marks {35, 72, 18, 90, 25}, remove all marks below 33. Loop over a COPY of the set so the original can be changed safely.

Solution:

```
marks = {35, 72, 18, 90, 25}

for m in marks.copy():
    if m < 33:
        marks.remove(m)

print(sorted(marks))
```

Output:

```
[35, 72, 90]
```

5. Set Operations

Q25. Union of Two Classes

BEGINNER

Students in Python batch {"Om", "Pia", "Raj"} and SQL batch {"Raj", "Sam"}. Find all students using union (|).

Solution:

```
python_batch = {"Om", "Pia", "Raj"}
sql_batch = {"Raj", "Sam"}

all_students = python_batch | sql_batch
print(sorted(all_students))
```

Output:

```
['Om', 'Pia', 'Raj', 'Sam']
```

Q26. Common Students

BEGINNER

Using the same two batches, find students who joined BOTH courses using intersection (&).

Solution:

```
python_batch = {"Om", "Pia", "Raj"}
sql_batch = {"Raj", "Sam"}

both = python_batch & sql_batch
print(both)
```

Output:

```
{'Raj'}
```

Q27. Only Python Students

INTERMEDIATE

Find students who are ONLY in the Python batch (difference -) and students in exactly one of the two batches (symmetric difference ^).

Solution:

```
python_batch = {"Om", "Pia", "Raj"}
sql_batch = {"Raj", "Sam"}

print(sorted(python_batch - sql_batch))
print(sorted(python_batch ^ sql_batch))
```

Output:

```
['Om', 'Pia']
['Om', 'Pia', 'Sam']
```

Q28. Missing Documents

INTERMEDIATE

Required documents are {"Aadhaar", "PAN", "Photo", "Marksheet"}. A student submitted {"Photo", "Aadhaar"}. Find the missing documents.

Solution:

```
required = {"Aadhaar", "PAN", "Photo", "Marksheet"}
submitted = {"Photo", "Aadhaar"}

missing = required - submitted
print(sorted(missing))
```

Output:

```
['Marksheet', 'PAN']
```

Q29. Common Friends Suggestion

ADVANCED

Riya's friends {"A", "B", "C", "D"} and Sam's friends {"C", "D", "E"}. Find mutual friends, and suggest to Riya the people Sam knows that Riya does not.

Solution:

```
riya = {"A", "B", "C", "D"}
sam = {"C", "D", "E"}

mutual = riya.intersection(sam)
print(sorted(mutual))

suggest = sam.difference(riya)
print(suggest)
```

Output:

```
['C', 'D']
{'E'}
```

Q30. Subset, Superset and Disjoint

ADVANCED

Check: is {"SQL"} a subset of a candidate's skills {"SQL", "Python", "Excel"}? Is the skill set a superset of {"Python", "Excel"}? Are {"Java"} and the skills disjoint (no common item)?

Solution:

```
skills = {"SQL", "Python", "Excel"}

print({"SQL"}.issubset(skills))
print(skills.issuperset({"Python", "Excel"}))
print(skills.isdisjoint({"Java"}))
```

Output:

```
True
True
True
```

6. Set Functions

Q31. len, min, max, sum

BEGINNER

Unique bill amounts are {250, 120, 980, 430}. Print len(), min(), max() and sum().

Solution:

```
bills = {250, 120, 980, 430}

print(len(bills))
print(min(bills))
print(max(bills))
print(sum(bills))
```

Output:

```
4
120
980
1780
```

Q32. sorted() Returns a List

BEGINNER

Sort the set {"Pune", "Agra", "Goa"} using sorted() in ascending and descending order. Print the type of the result.

Solution:

```
cities = {"Pune", "Agra", "Goa"}

print(sorted(cities))
print(sorted(cities, reverse=True))
print(type(sorted(cities)))
```

Output:

```
['Agra', 'Goa', 'Pune']
['Pune', 'Goa', 'Agra']
<class 'list'>
```

Q33. copy() vs Assignment

INTERMEDIATE

Make a = {1, 2, 3}, b = a and c = a.copy(). Add 4 to b. Print all three sets to see which one changed.

Solution:

```
a = {1, 2, 3}
b = a
c = a.copy()

b.add(4)
print(a)
print(b)
print(c)
```

Output:

```
{1, 2, 3, 4}
{1, 2, 3, 4}
{1, 2, 3}
```

Q34. Update Methods

INTERMEDIATE

Start with x = {1, 2, 3, 4}. Apply intersection_update({2, 3, 5}) and print x. Then on y = {1, 2, 3} apply symmetric_difference_update({3, 4}).

Solution:

```
x = {1, 2, 3, 4}
x.intersection_update({2, 3, 5})
print(x)

y = {1, 2, 3}
y.symmetric_difference_update({3, 4})
print(y)
```

Output:

```
{2, 3}
{1, 2, 4}
```

Q35. Find Duplicate Entries

ADVANCED

Roll numbers entered are [12, 15, 12, 18, 15, 20]. Using a set, print the roll numbers that were entered more than once.

Solution:

```
entries = [12, 15, 12, 18, 15, 20]
seen = set()
duplicates = set()

for r in entries:
    if r in seen:
        duplicates.add(r)
    else:
        seen.add(r)

print(sorted(duplicates))
```

Output:

```
[12, 15]
```

Q36. Compare Two Product Catalogues

ADVANCED

Old catalogue {"P1", "P2", "P3", "P4"} and new catalogue {"P2", "P4", "P5"}. Print products added, removed and unchanged using set methods.

Solution:

```
old = {"P1", "P2", "P3", "P4"}
new = {"P2", "P4", "P5"}

print(sorted(new.difference(old)))
print(sorted(old.difference(new)))
print(sorted(old.intersection(new)))
```

Output:

```
['P5']
['P1', 'P3']
['P2', 'P4']
```

7. Frozenset

Q37. Create a Frozenset

BEGINNER

Primary colours never change. Create a frozenset of "Red", "Blue", "Yellow" and print its type and sorted values.

Solution:

```
primary = frozenset(["Red", "Blue", "Yellow"])

print(type(primary))
print(sorted(primary))
```

Output:

```
<class 'frozenset'>
['Blue', 'Red', 'Yellow']
```

Q38. Frozenset Cannot Change

BEGINNER

Try to add "Green" to the frozenset of primary colours inside try-except and print the error.

Solution:

```
primary = frozenset(["Red", "Blue", "Yellow"])

try:
    primary.add("Green")
except AttributeError as e:
    print(e)
```

Output:

```
'frozenset' object has no attribute 'add'
```

Q39. Operations Still Work

INTERMEDIATE

Weekend = frozenset({"Sat", "Sun"}), holidays = {"Sun", "Mon"}. Find the union and intersection.

Solution:

```
weekend = frozenset({"Sat", "Sun"})
holidays = {"Sun", "Mon"}

print(sorted(weekend | holidays))
print(weekend & holidays)
```

Output:

```
['Mon', 'Sat', 'Sun']
frozenset({'Sun'})
```

Q40. Frozenset as Dictionary Key

INTERMEDIATE

Store the price of combos where the order of items does not matter: frozenset({"Burger", "Coke"}) = 199. Look up the price using frozenset({"Coke", "Burger"}).

Solution:

```
combos = {
    frozenset({"Burger", "Coke"}): 199,
    frozenset({"Pizza", "Pepsi"}): 349
}

print(combos[frozenset({"Coke", "Burger"})])
```

Output:

```
199
```

Q41. Set of Sets

ADVANCED

Store study groups as a set of frozensets: {"Om", "Pia"} and {"Raj", "Sam"}. Try the same with normal sets inside try-except.

Solution:

```
groups = {frozenset({"Om", "Pia"}), frozenset({"Raj", "Sam"})}
print(len(groups))

try:
    bad = {{"Om", "Pia"}}
except TypeError as e:
    print(e)
```

Output:

```
2
unhashable type: 'set'
```

Q42. Unique Pairs of Players

ADVANCED

From players ["A", "B", "C"] create all unique match pairs where (A, B) and (B, A) are the same match. Use frozensets in a set.

Solution:

```
players = ["A", "B", "C"]
matches = set()

for p1 in players:
    for p2 in players:
        if p1 != p2:
            matches.add(frozenset({p1, p2}))

print(len(matches))
for m in sorted(sorted(x) for x in matches):
    print(m)
```

Output:

```
3
['A', 'B']
['A', 'C']
['B', 'C']
```

8. Set Comprehension

Q43. Squares as a Set

BEGINNER

Create a set of squares of numbers 1 to 6 using set comprehension.

Solution:

```
squares = {n * n for n in range(1, 7)}
print(sorted(squares))
```

Output:

```
[1, 4, 9, 16, 25, 36]
```

Q44. Unique Lengths

BEGINNER

Find the unique word lengths in ["cat", "dog", "tiger", "lion", "bear"] using set comprehension.

Solution:

```
animals = ["cat", "dog", "tiger", "lion", "bear"]

lengths = {len(a) for a in animals}
print(sorted(lengths))
```

Output:

```
[3, 4, 5]
```

Q45. Even Numbers Only

INTERMEDIATE

From the list [4, 7, 4, 10, 15, 10, 22], create a set of unique even numbers using a condition.

Solution:

```
numbers = [4, 7, 4, 10, 15, 10, 22]

evens = {n for n in numbers if n % 2 == 0}
print(sorted(evens))
```

Output:

```
[4, 10, 22]
```

Q46. Normalise Email Domains

INTERMEDIATE

Emails are ["a@Gmail.com", "b@gmail.com", "c@Yahoo.in", "d@yahoo.in"]. Create a set of unique domains in lowercase.

Solution:

```
emails = ["a@Gmail.com", "b@gmail.com",
          "c@Yahoo.in", "d@yahoo.in"]

domains = {e.split("@")[1].lower() for e in emails}
print(sorted(domains))
```

Output:

```
['gmail.com', 'yahoo.in']
```

Q47. Vowels Used in a Sentence

ADVANCED

Find which vowels are used in "Python programming is fun" and which vowels are NOT used, using set comprehension and difference.

Solution:

```
sentence = "Python programming is fun"

used = {ch for ch in sentence.lower() if ch in "aeiou"}
print(sorted(used))

not_used = set("aeiou") - used
print(sorted(not_used))
```

Output:

```
['a', 'i', 'o', 'u']
['e']
```

Q48. Common Items in Many Orders

ADVANCED

Three orders contain ["milk", "bread", "eggs"], ["bread", "milk", "tea"], ["milk", "bread", "jam"]. Find items present in ALL orders by intersecting sets built with comprehension.

Solution:

```
orders = [["milk", "bread", "eggs"],
          ["bread", "milk", "tea"],
          ["milk", "bread", "jam"]]

sets = [{item for item in o} for o in orders]
common = sets[0]
for s in sets[1:]:
    common = common & s

print(sorted(common))
```

Output:

```
['bread', 'milk']
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026